

Sistemas Operativos

Computación - UNRC

Práctico 2: Compilación y linking.

Dados los programas `main.c` y `hello.c` dados en el capítulo <https://marceloarroyo.gitlab.io/cursos/SO/index.html#/toolchain> de las notas del curso.

1. Realizar los comandos de compilación de `hello.c` para generar:
 - a. El archivo resultante del pre-procesamiento.
 - b. El archivo en *assembly*.
 - c. El archivo objeto (`hello.o`).
2. Compilar el programa `main.c` para obtener el archivo objeto `main.o`.
3. Analizar¹ el código assembly (mediante la opción de disassembly) de ambos archivos objeto. En particular determinar:
 - a. Ver el tipo de cada archivo objeto (Ej: `objdump -a`).
 - b. Listar las secciones (headers) de cada archivo objeto (ver opción `-h`). Determinar el tamaño de la sección de código (`.text`) y datos globales (`.data`).
 - c. Ver el contenido de cada sección de los archivos objeto (`objdump -s`).
 - d. En qué dirección comienzan las funciones `main()` y `hello()` en cada archivo objeto.
 - e. Determinar en qué área de memoria está almacenado el string (literal) apuntado por `hi` y la dirección de memoria de `hi`.
 - f. Listar y analizar la *tabla de símbolos* (`objdump -t`) y las de *reubicación* (`objdump -r`) en ambos archivos objeto.
 - g. Las instrucciones `call` en `main()`. ¿Están resueltos los operandos de estas instrucciones?
 - h. Cómo retornan los valores las funciones.
4. Generar el ejecutable *myprog* desde `main.o` y `hello.o`. ¿Se recompilaron los programas? Ejecutar `gcc` con `-v` para ver los pasos de compilación.
5. Analizar el código assembly de *myprog*.
 - a. Determinar las direcciones de la función `main()` y `hello()`.
 - b. Analizar si la invocación a `hello()` desde `main()` fue resuelta por el linker.

¹ En GNU-Linux usar `objdump` o `readelf`, comúnmente se encuentran en el paquete *binutils* (Ubuntu y otras distros. En MAC-OS, usar `otool`).

- c. ¿Cómo se resolvió estáticamente la invocación a `printf()`?
- 6. Modificar el programa `main.c` para determinar en qué dirección de memoria se asigna a variables locales.
- 7. Crear otro módulo (ej: `hello2.c`) con otra función a invocar desde `main()`.
 - a. Crear una biblioteca estática `libhello.a` con los archivos objeto `hello.o` y `hello2.o`. (ver el comando `ar`)
 - b. Listar los módulos contenidos en la biblioteca.
 - c. Compilar el programa `myprog` enlazando con la biblioteca `libhello.a` y luego ejecutarlo.
- 8. Compilar `hello.c` para generar una biblioteca de enlazado dinámico. Usar la opción `-shared` en el comando `gcc`. El nombre de la biblioteca debería ser (`libhello.so` en Linux o `libhello.dylib` en MacOS).
 - a. Modificar `main.c` para que cargue dinámicamente (en ejecución) la biblioteca (ver `dlopen()`), resuelva la dirección de la función `hello()` (ver `dlsym()`) e invocarla. Usar el programa dado en las notas del curso.
 - b. Compilar la nueva versión de `main.c` como `myprog2`.
 - c. Analizar el código de `myprog2`. Notar que no está el código de `hello.o`.
 - d. Analizar `myprog2` con `objdump` la sección de enlazado dinámico (opción `-T`). Deberían aparecer las bibliotecas dinámicas necesarias por `myprog2`.
 - e. Ejecutar `myprog2`.
- 9. Escribir un archivo `Makefile` para compilar `main.c` y `hello.c` (originales) para generar `myprog` con el comando `make`.